

Load Balancer Project

Alaa Abo Mokh

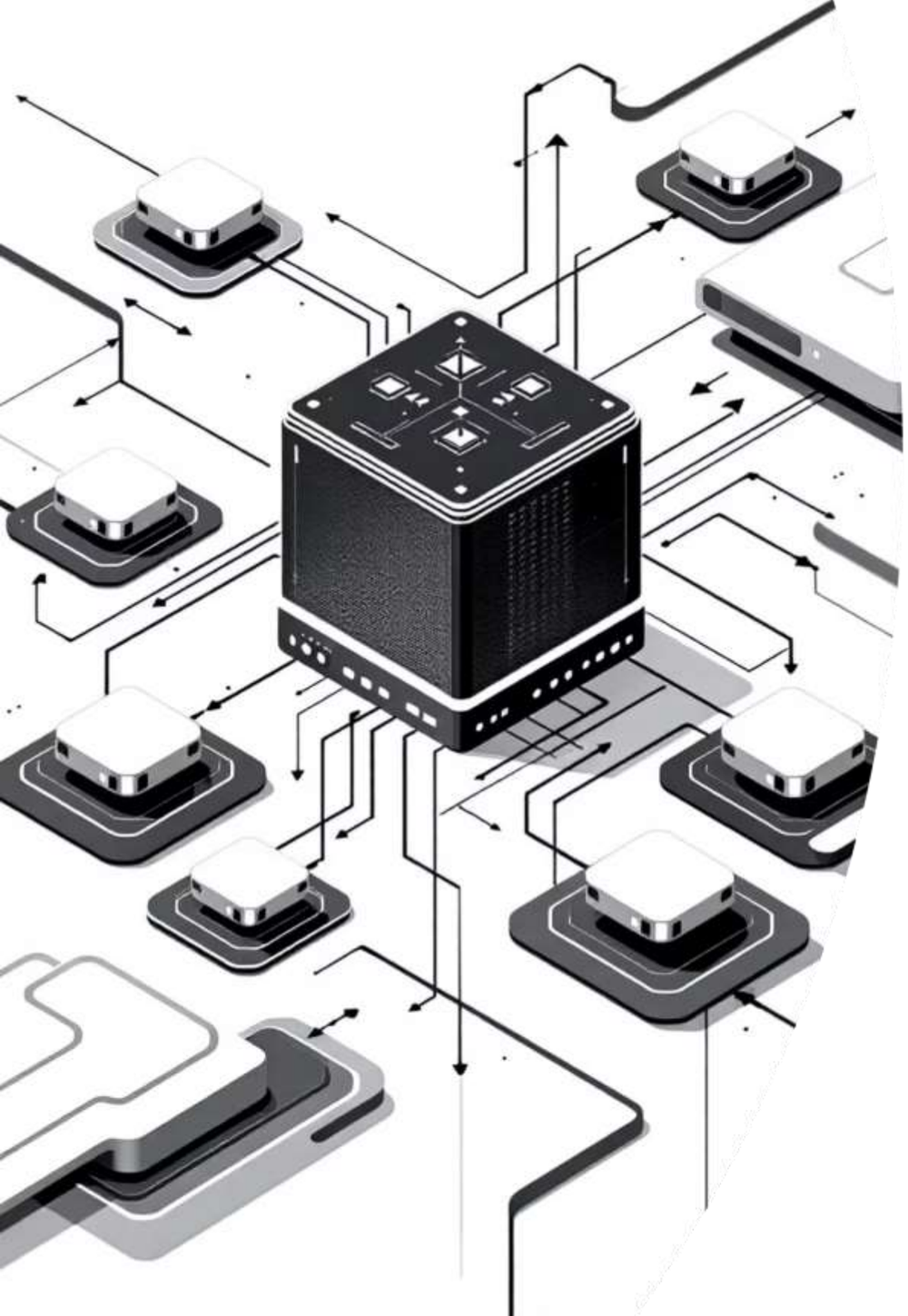
Mayyas Iktilat

Adan Abo Mokh

Project Goals

The goal of this project is to compare IP Hash and Least Connections algorithms under different workloads and measure performance (total completion time) to understand which policy provides better load distribution and client experience.



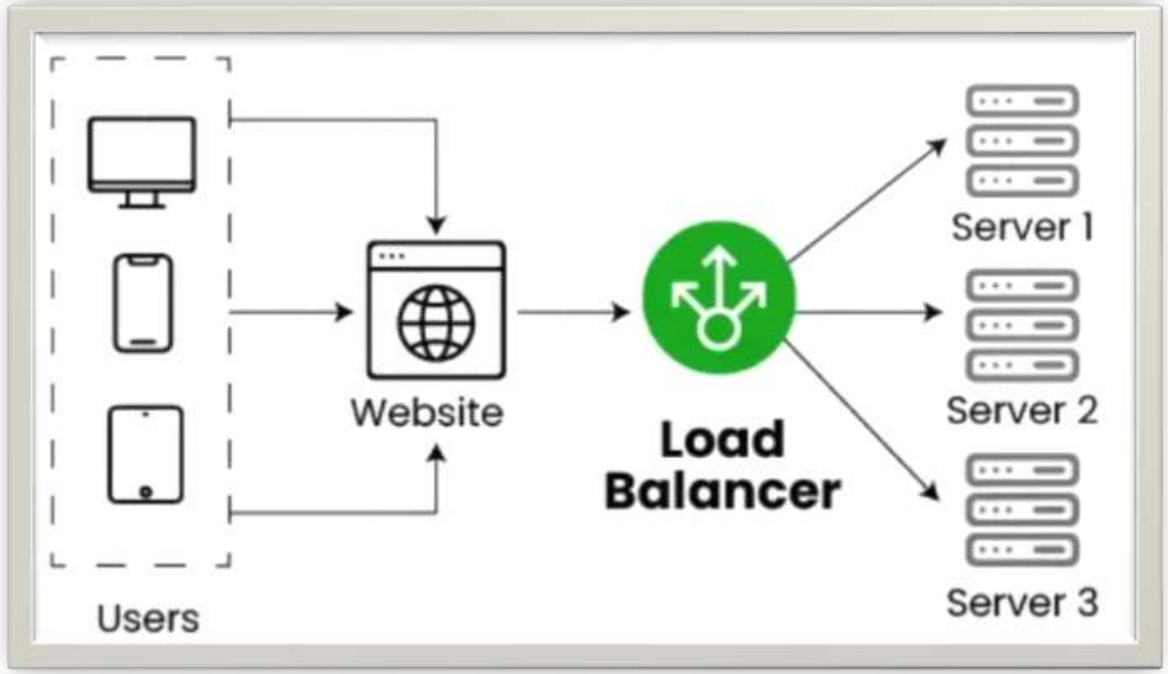


Load Balancer

Short Overview of Concepts & Algorithms

What is a Load Balancer?

A load balancer is a networking device or software application that distributes and balances the incoming traffic among the servers to provide high availability, efficient utilization of servers and high performance.



Key Characteristics

Traffic Distribution

Directs incoming requests to the most suitable backend server.

High Availability

Maintains service uptime by redirecting traffic away from unhealthy servers.

Scalability

Enables seamless addition or removal of servers to handle varying loads.

Optimization

Enhances application response times and throughput.

Health Monitoring

Continuously checks the status of backend servers to ensure operational readiness.

Load Balancing Algorithms

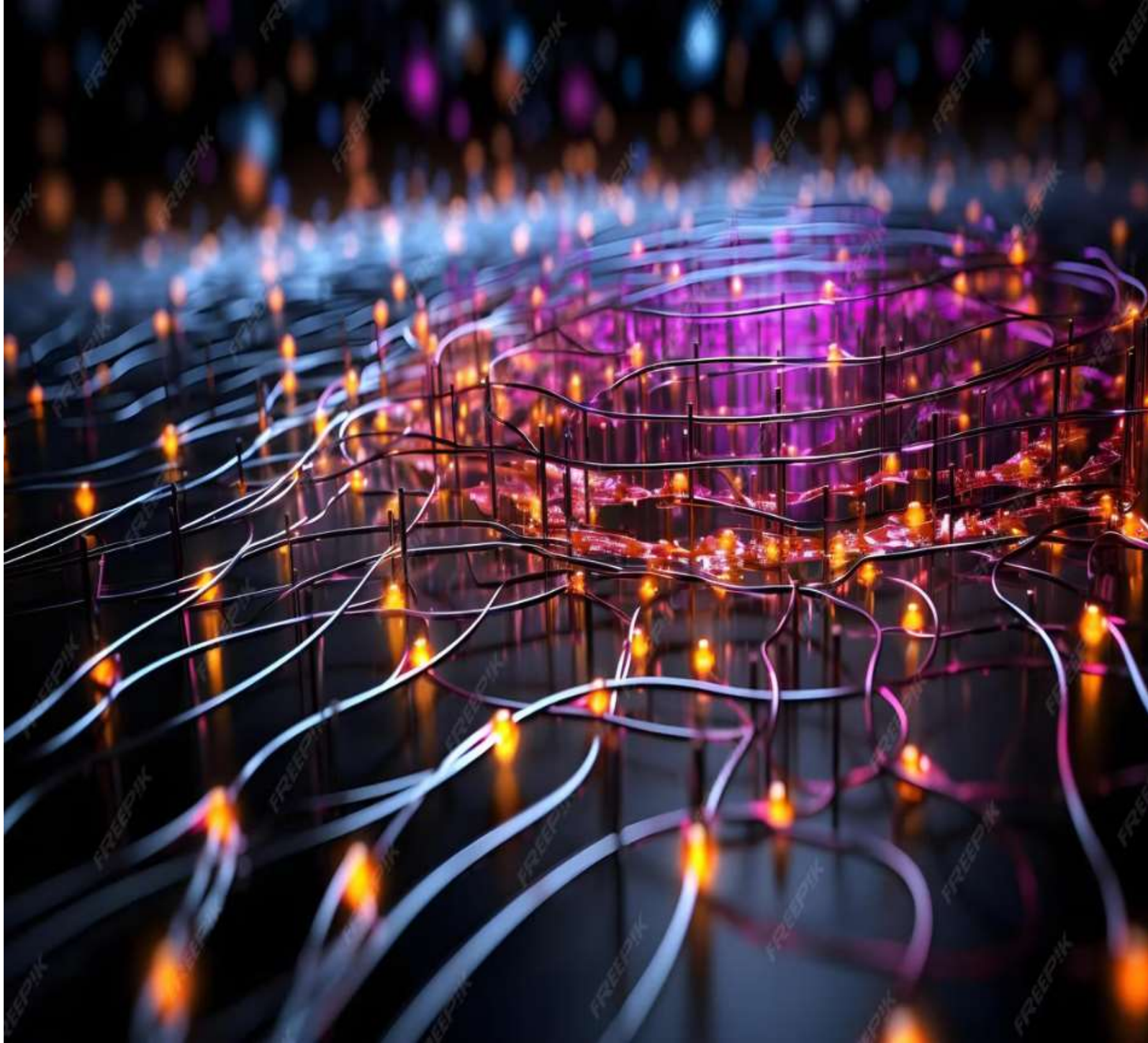
Static Algorithms

Fixed methods that distribute requests without considering current server load.

Dynamic Algorithms

Intelligent methods that adapt distribution based on real time server conditions (e.g., CPU, memory, response times).

Load Balancing Static Algorithm

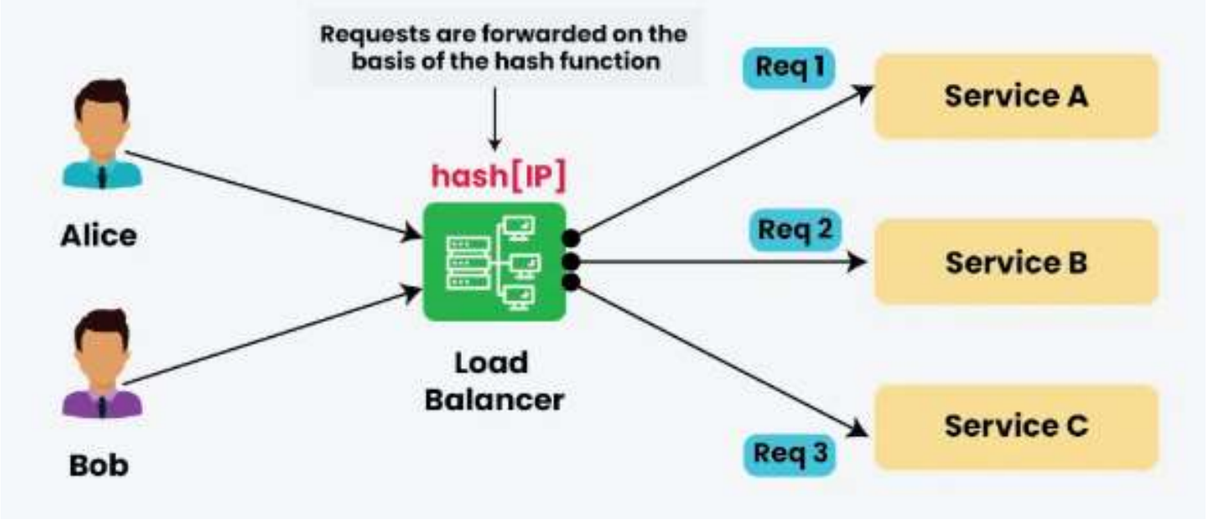


Source IP Hash

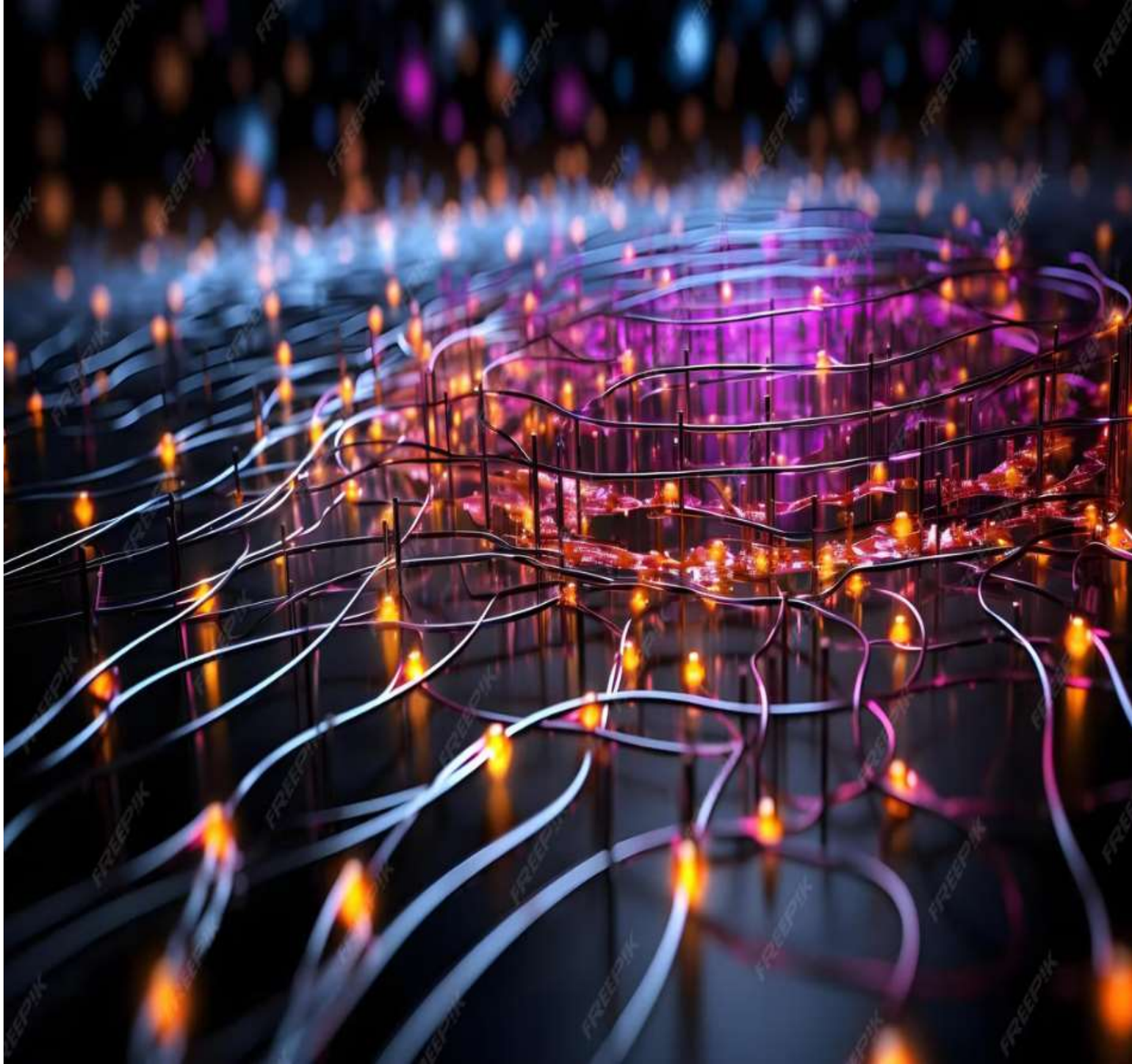
The Source IP Hash algorithm provides a predictable and consistent routing mechanism by mapping a client's IP address to a specific server.

How it Works

- A hash function is applied to the client's source IP address.
- This hash value then determines which server will handle the request.
- Crucially, the same client IP will always be routed to the same server, ensuring session persistence.



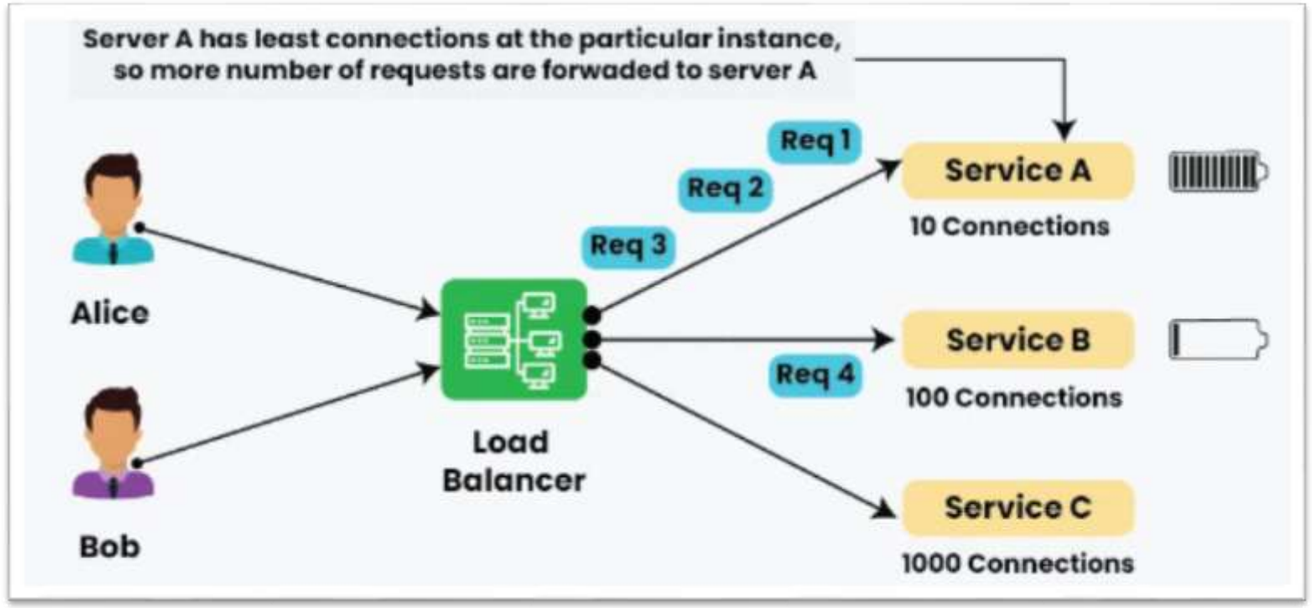
Load Balancing Dynamic Algorithm



Least Connections

Real-time Adaptation

Selects the server with the fewest active connections, ensuring immediate load balancing based on current state.



Variable Request Duration

Excellent for applications where individual request processing times can vary significantly, preventing bottlenecks.

Fluctuating Traffic

Dynamically adjusts to sudden spikes or dips in traffic, maintaining optimal performance across all active servers.

Ns3 vs Mininet

Feature / Aspect	Ns-3	Mininet
Type of Environment	Full network simulator	Realistic emulator
Accuracy of Results	Highly reproducible, fully controlled simulations	Results depend on host machine performance (less deterministic)
Protocol Stack	Uses simulated TCP/IP stack written in ns-3	Uses the real Linux TCP/IP stack
Performance	Very fast simulations even with many nodes	Can be slow for large topologies because everything runs on one machine
Control Over Packets	Full control (headers, queues, events, timing)	Limited — system level behavior controlled by Linux
Best For	Academic research, algorithm evaluation, graphs, reproducibility	Emulating real networks, SDN, datacenter behavior

project's stages

1

Environment Setup

Set up Mininet, prepare the development environment, and initialize the project structure.

2

Topology Setup

Build the network: clients → load balancer → servers, using controlled link parameters.

3

Load Balancer Implementation

Implement request handling logic and integrate algorithms such as Least Connections and IP Hash.

4

Traffic Simulation

Generate realistic client traffic and simulate server responses to evaluate system behavior.

5

Performance Measurement

Collect metrics such as total completion time.

6

Repetition & Stability

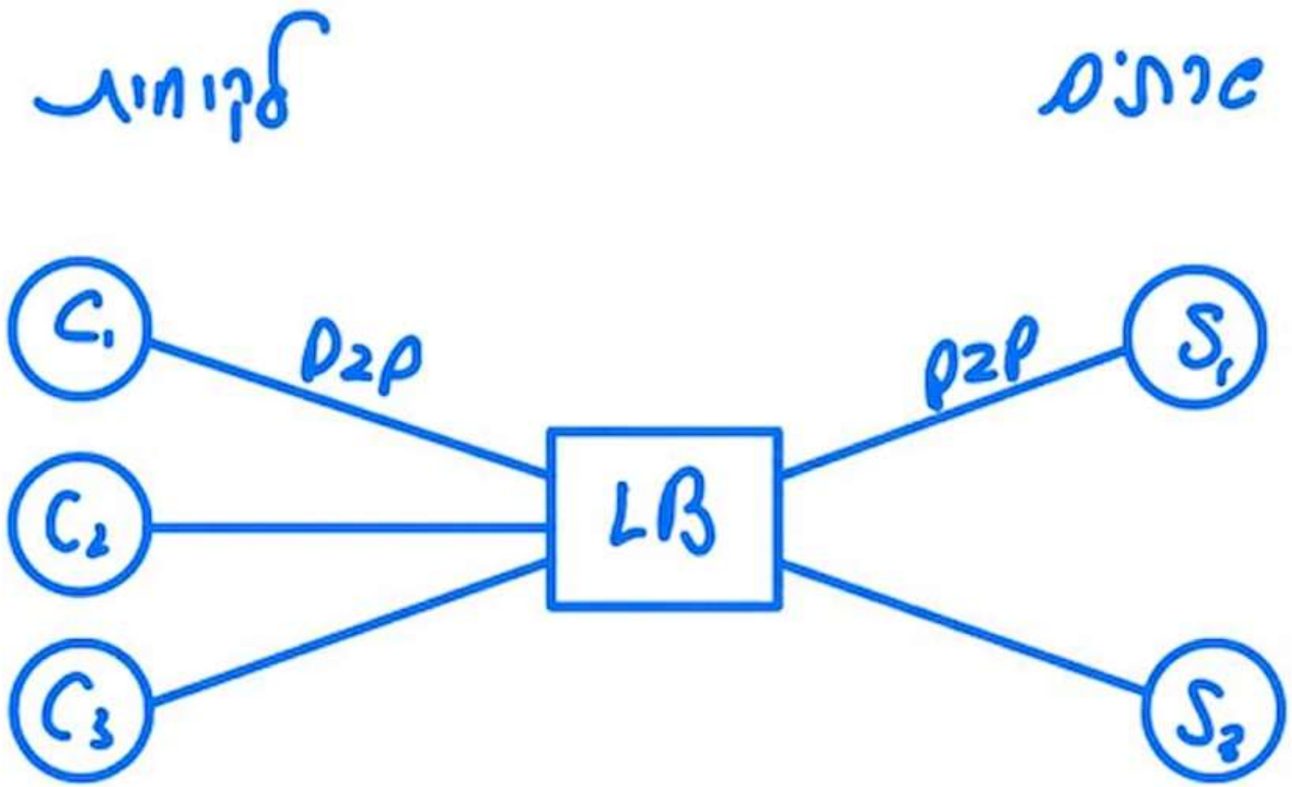
Automate multi run scenarios using scripts to ensure stable and reproducible results.

7

Results Visualization

Generate plots and graphs to present the system’s performance and compare load balancing algorithms.

Topology :

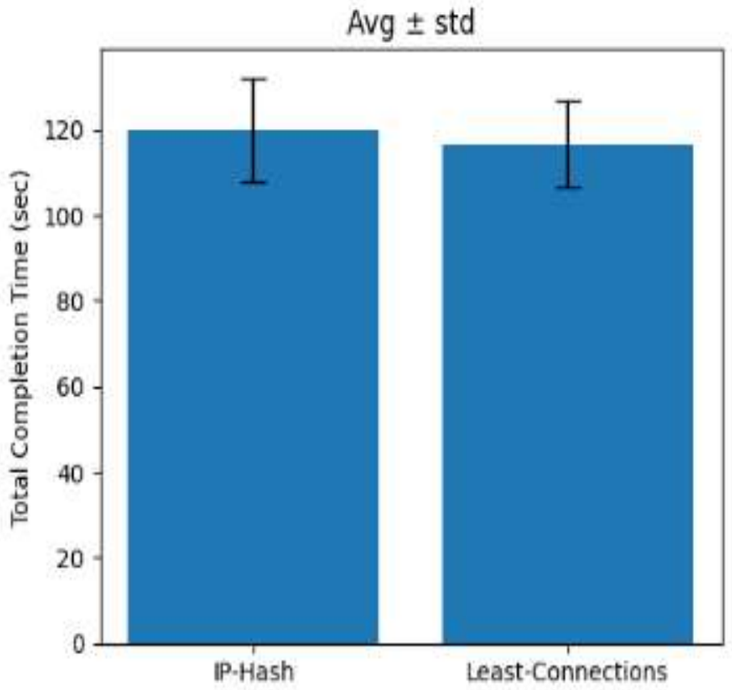


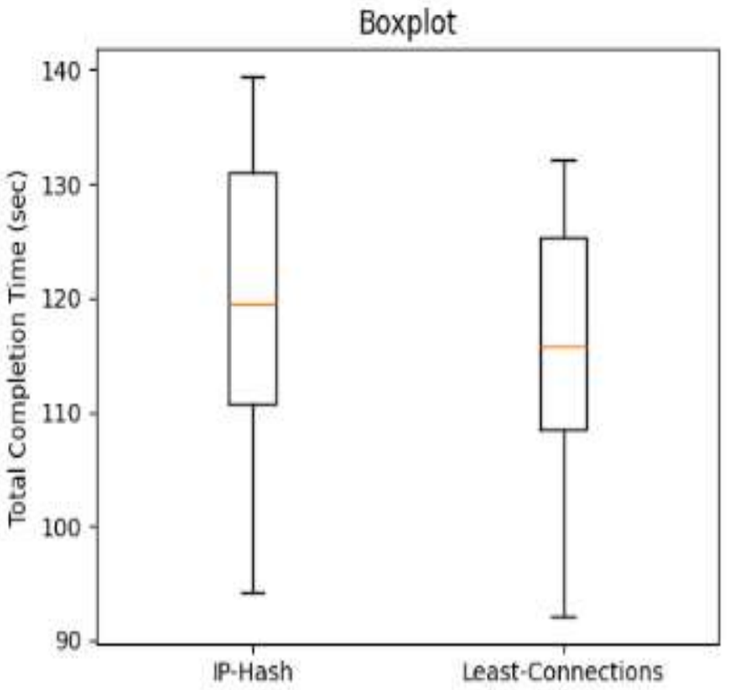
What we expect ?

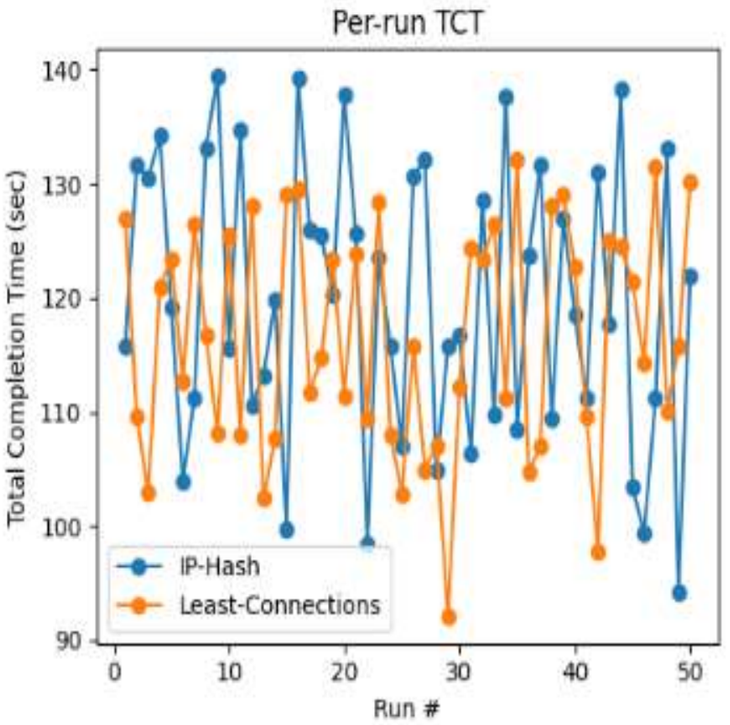
Least Connections
will probably give the best performance when traffic is not evenly distributed.

IP-Hash
will not give the best load distribution, but it is useful for keeping the same client on the same server.

Baseline – Symmetric servers

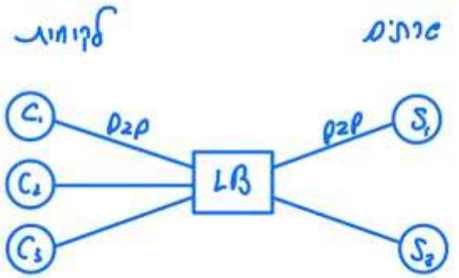




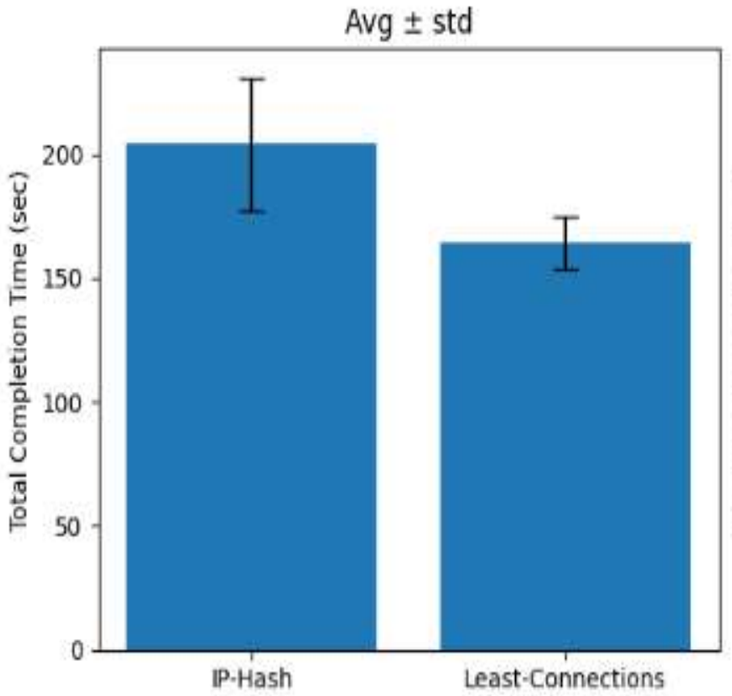


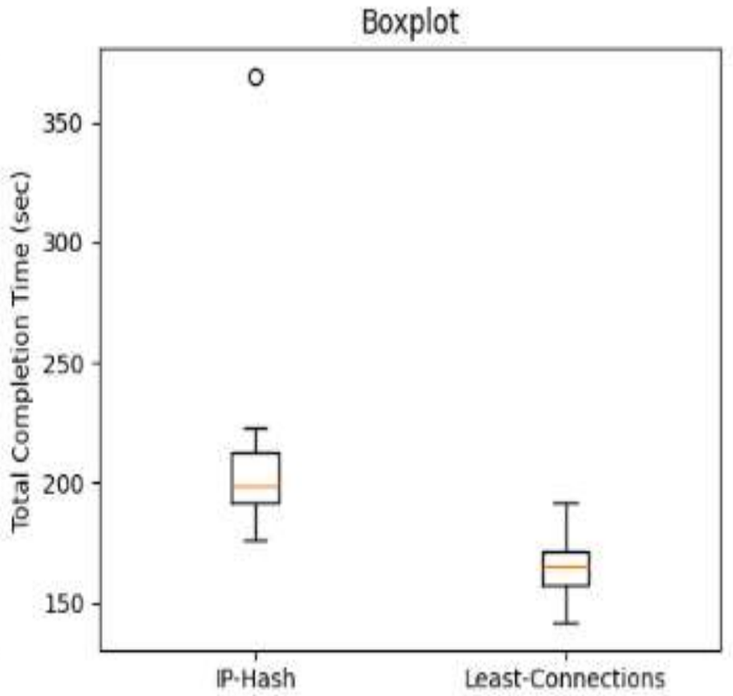

```
SERVICE_TIMES = {
  "M": 0.15,  # Music
  "V": 0.40,  # Video
  "P": 0.08,  # Photo
}
```

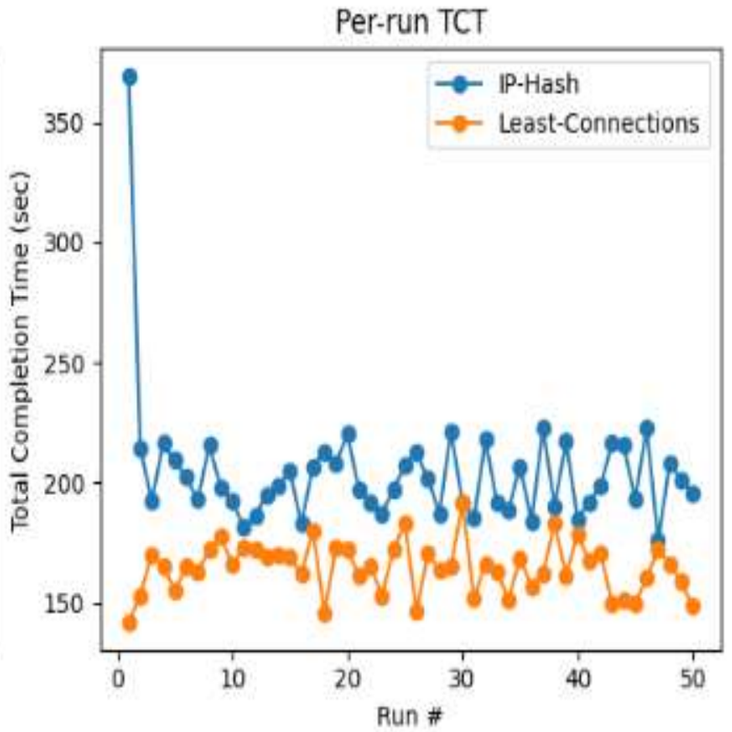
Results:-
GAP_MIN=0.05, GAP_MAX=0.2
Average IP-Hash TCT: 119.891s
Std IP-Hash TCT: 12.264s
Average Least-Connections TCT: 116.662s
Std Least-Connections TCT: 10.069s



Heterogeneous setup – Single slow server







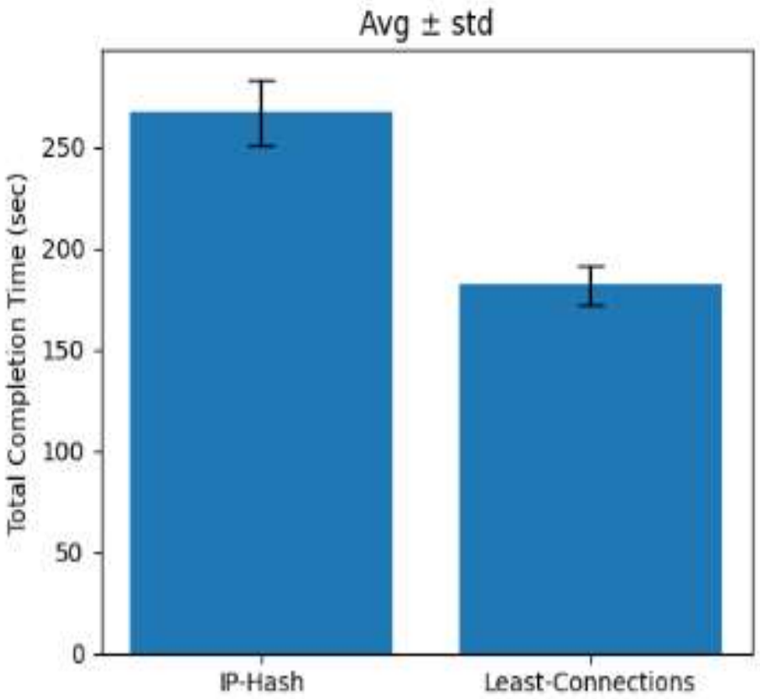
Slow server:
SERVICE_TIMES = {
"M": 0.30,
"V": 0.80,
"P": 0.20
}

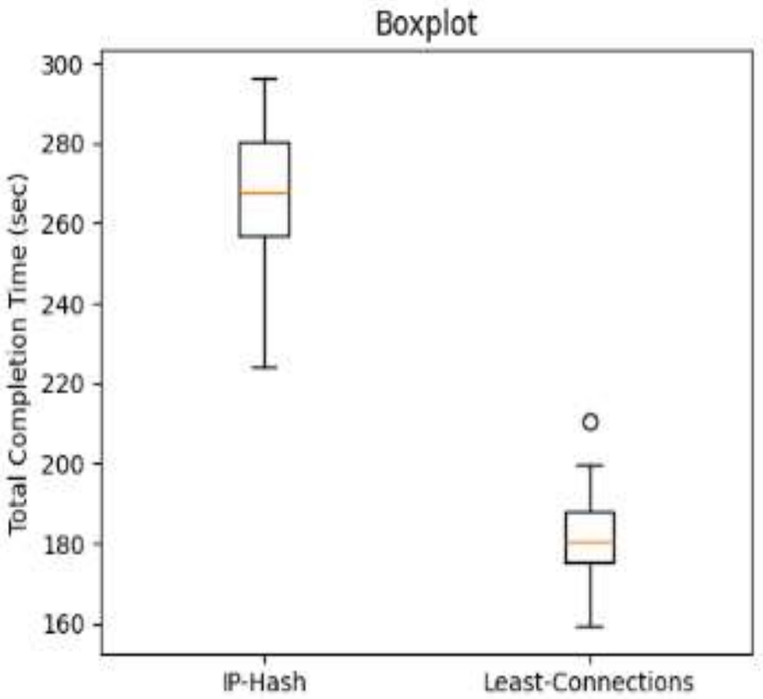
Fast server:
SERVICE_TIMES = {
"M": 0.10,
"V": 0.30,
"P": 0.05
}

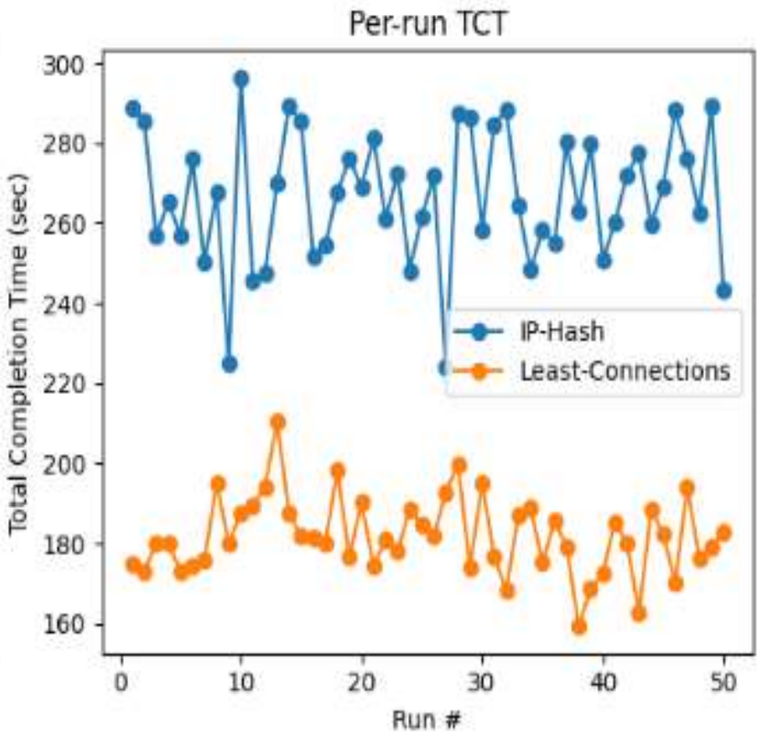
Results:-
GAP_MIN=0.05, GAP_MAX=0.2
Average IP-Hash TCT: 204.078s
Std IP-Hash TCT: 26.876s
Average Least-Connections TCT: 164.542s
Std Least-Connections TCT: 10.609s

Client	Hash result	Server
10.0.0.1	1	10.0.0.4
10.0.0.2	0	10.0.0.3
10.0.0.5	1	10.0.0.4

Extreme heterogeneity – Highly imbalanced servers







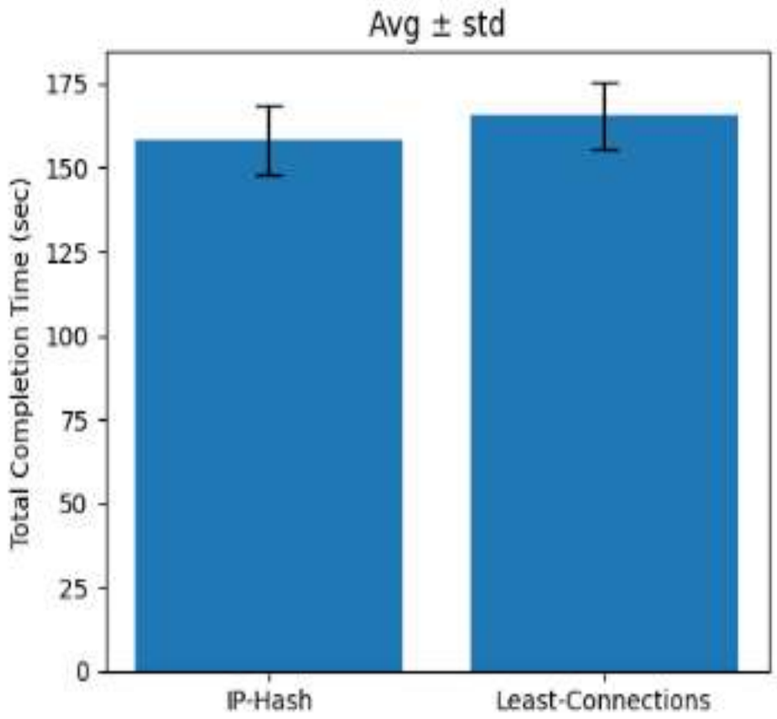
Slow server:
SERVICE_TIMES = {
"M": 0.50,
"V": 1.20,
"P": 0.30
}

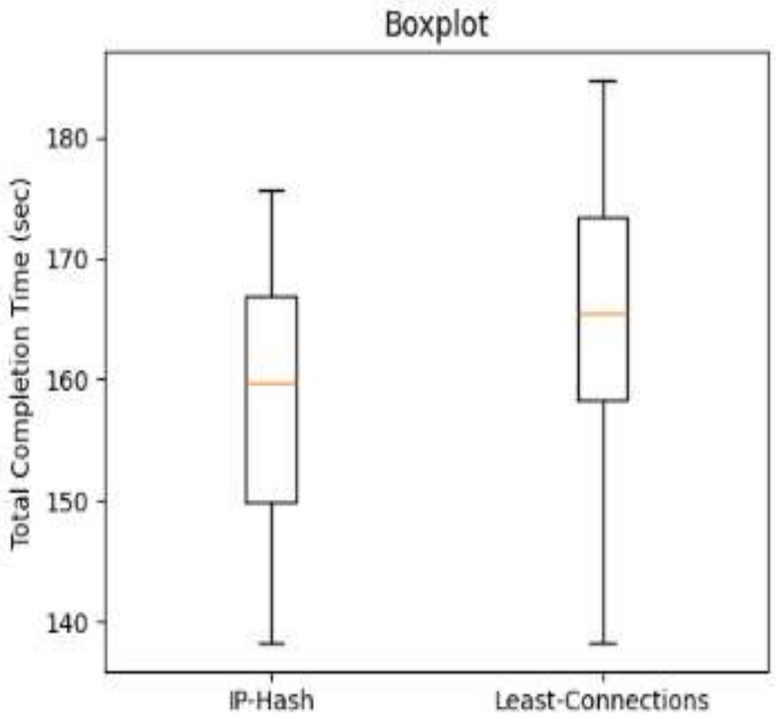
Fast server:
SERVICE_TIMES = {
"M": 0.10,
"V": 0.30,
"P": 0.05
}

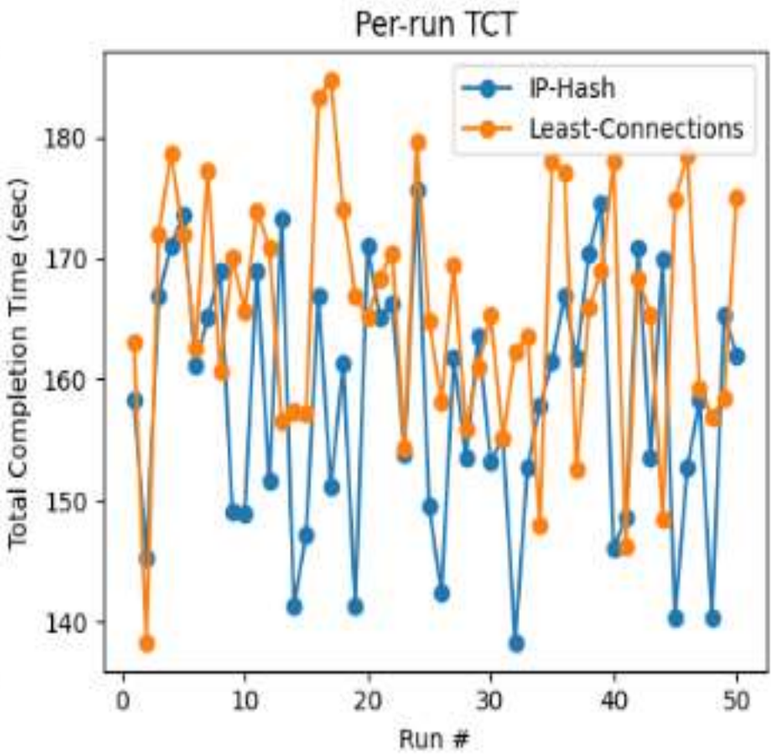
Results:-
GAP_MIN=0.05, GAP_MAX=0.2
Average IP-Hash TCT: 266.994s
Std IP-Hash TCT: 16.523s
Average Least-Connections TCT: 181.947s
Std Least-Connections TCT: 9.674s

Client	Hash result	Server
10.0.0.1	1	10.0.0.4
10.0.0.2	0	10.0.0.3
10.0.0.5	1	10.0.0.4

Mapping-sensitive scenario – Favorable IP distribution







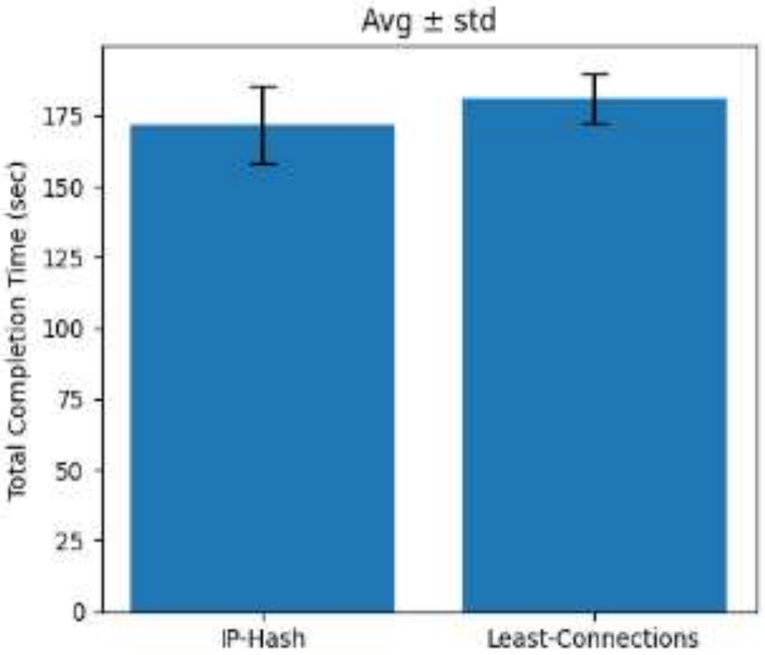
Slow server:
SERVICE_TIMES = {
"M": 0.30,
"V": 0.80,
"P": 0.20
}

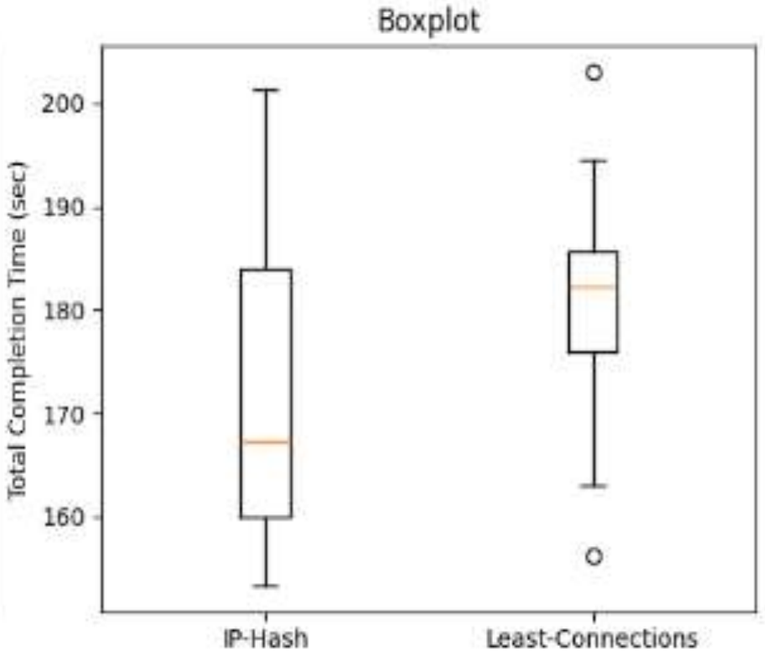
Fast server:
SERVICE_TIMES = {
"M": 0.10,
"V": 0.30,
"P": 0.05
}

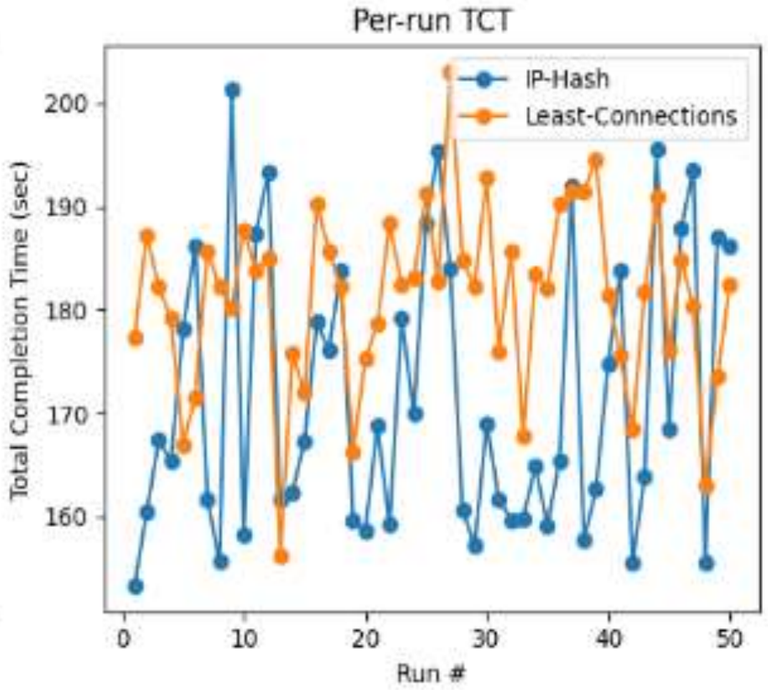
Results:-
GAP_MIN=0.05, GAP_MAX=0.2
Average IP-Hash TCT: 158.303s
Std IP-Hash TCT: 10.559s
Average Least-Connections TCT: 165.597s
Std Least-Connections TCT: 10.158s

Client	Hash result	Server
10.0.0.1	1	10.0.0.4
10.0.0.2	0	10.0.0.3
10.0.0.5	1	10.0.0.4

Favorable IP distribution with slower server







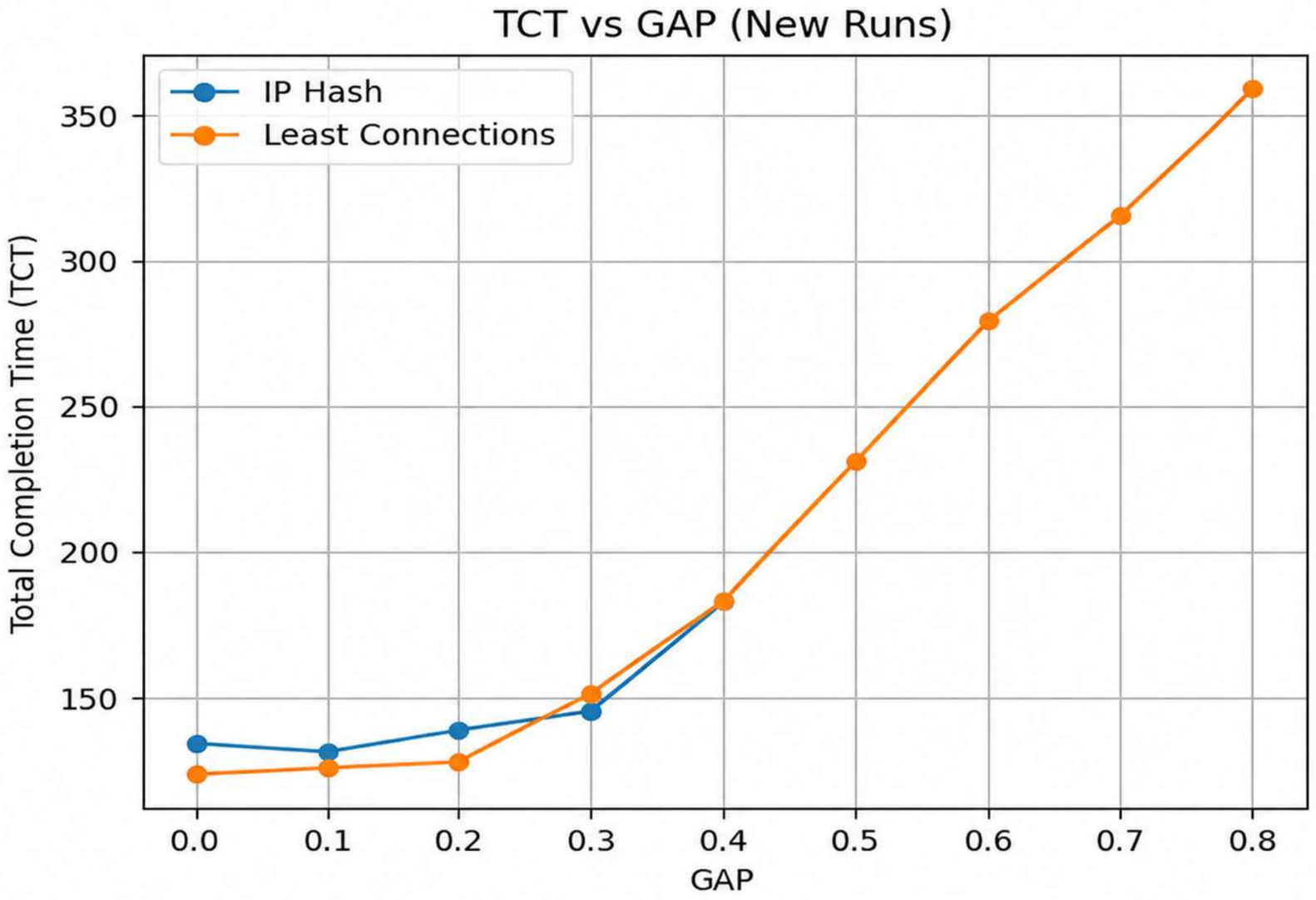
Slow server:
SERVICE_TIMES = {
"M": 0.50,
"V": 1.20,
"P": 0.30
}

Fast server:
SERVICE_TIMES = {
"M": 0.10,
"V": 0.30,
"P": 0.05
}

Results:-
GAP_MIN=0.05, GAP_MAX=0.2
Average IP-Hash TCT: 171.697s
Std IP-Hash TCT: 13.546s
Average Least-Connections TCT: 181.296s
Std Least-Connections TCT: 8.748s

Client	Hash result	Server
10.0.0.1	1	10.0.0.4
10.0.0.2	0	10.0.0.3
10.0.0.5	1	10.0.0.4

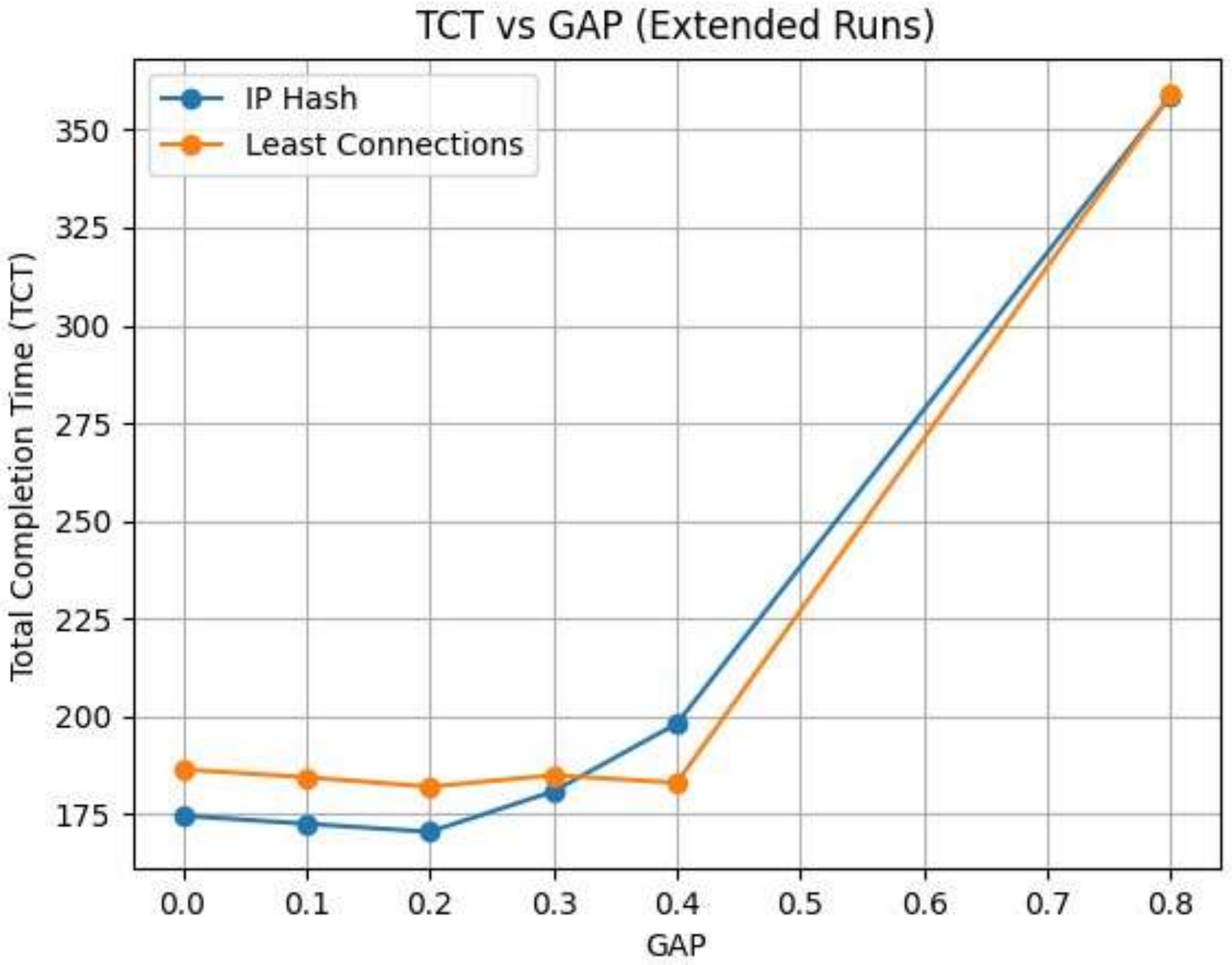
Symmetric servers



Fast server:

```
SERVICE_TIMES = {
  "M": 0.10,
  "V": 0.30,
  "P": 0.05
}
```

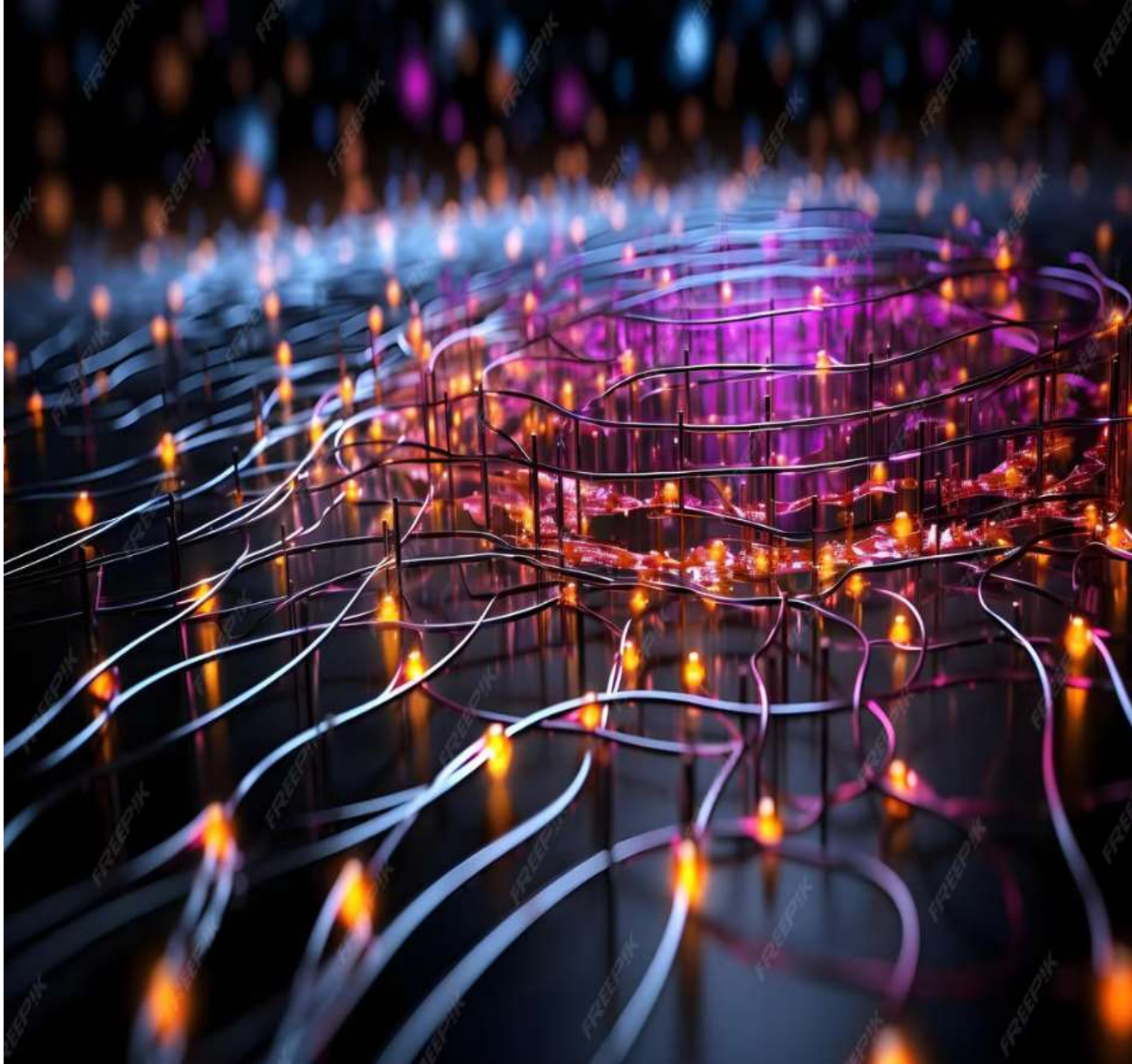
Favorable IP distribution



Fast server:

```
SERVICE_TIMES = {
  "M": 0.10,
  "V": 0.30,
  "P": 0.05
}
```

Final Results

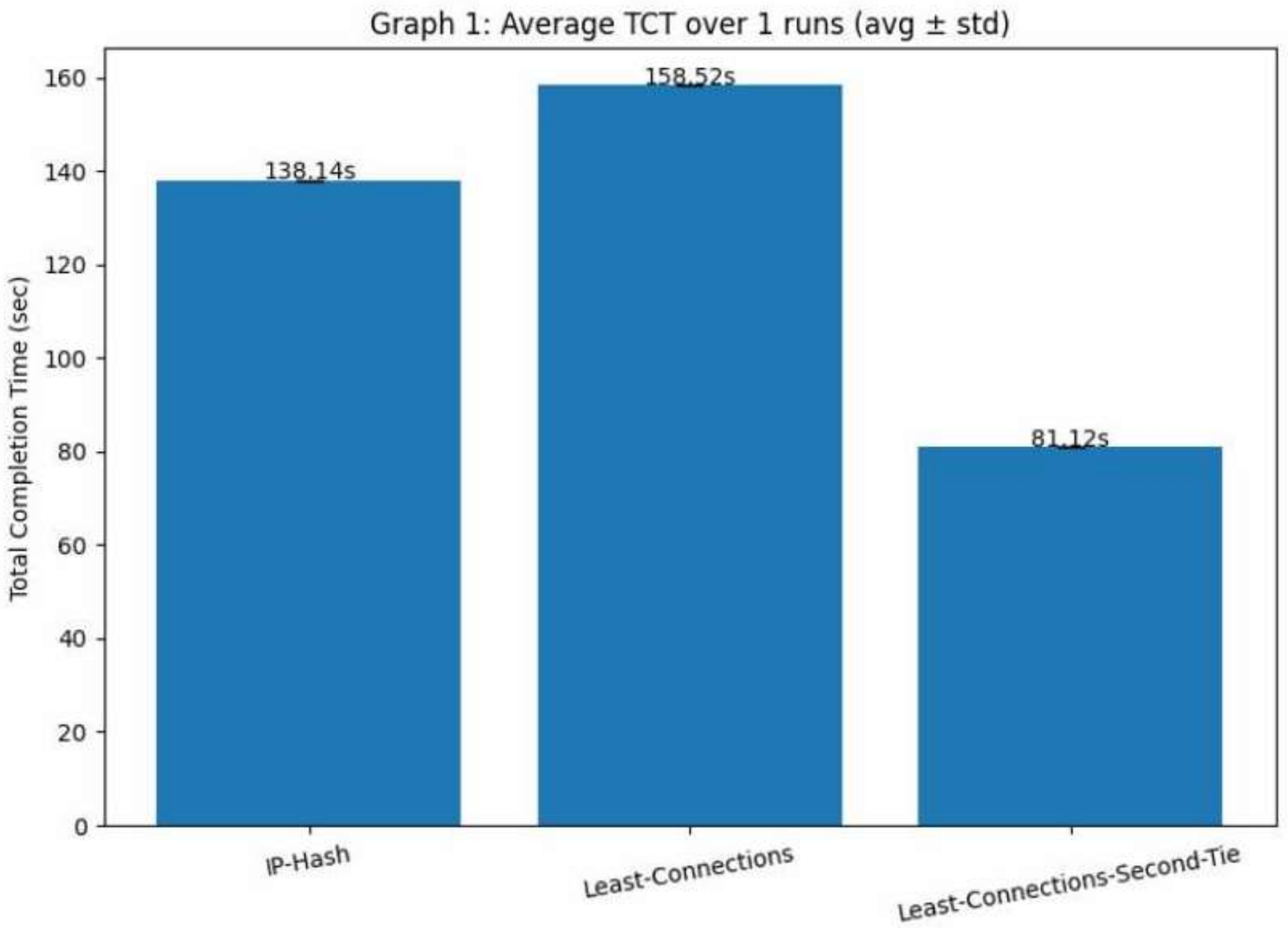


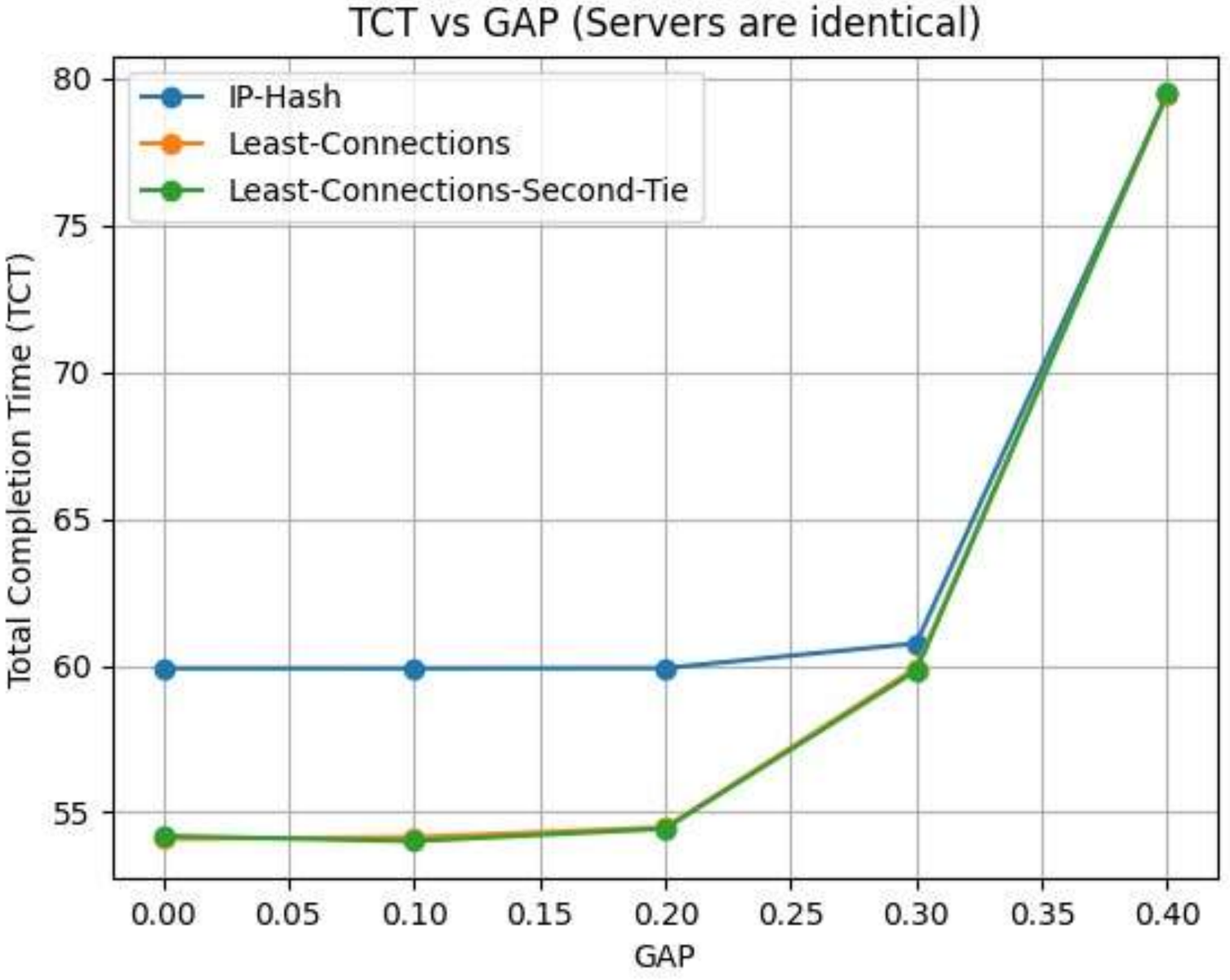
algorithm=IPHash
h3=192
h4=384
total=576

algorithm=LeastConn
h3=223
h4=353
total=576

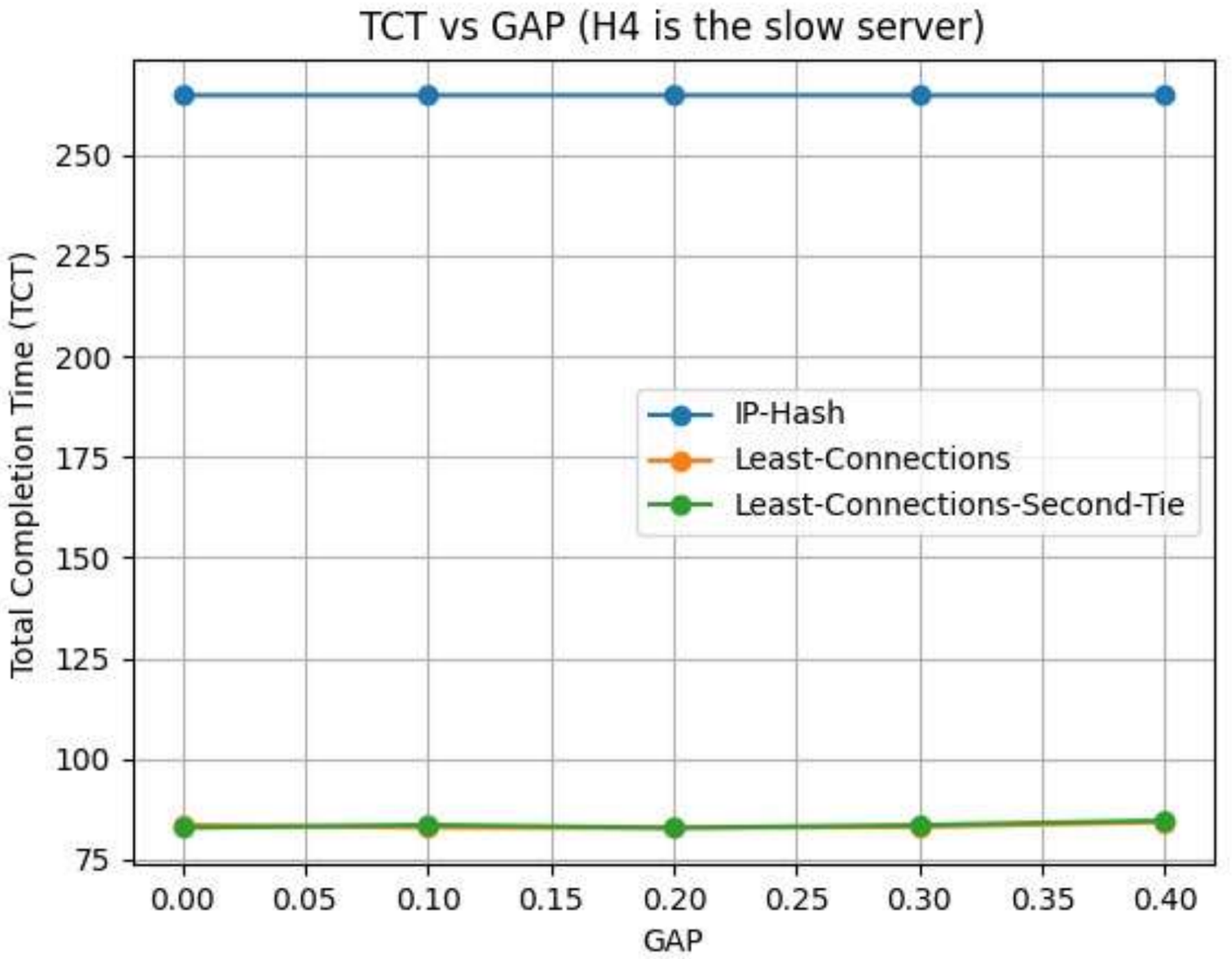
algorithm=LeastConnSecondTie
h3=81
h4=495
total=576

GAP=0
H4 is fast

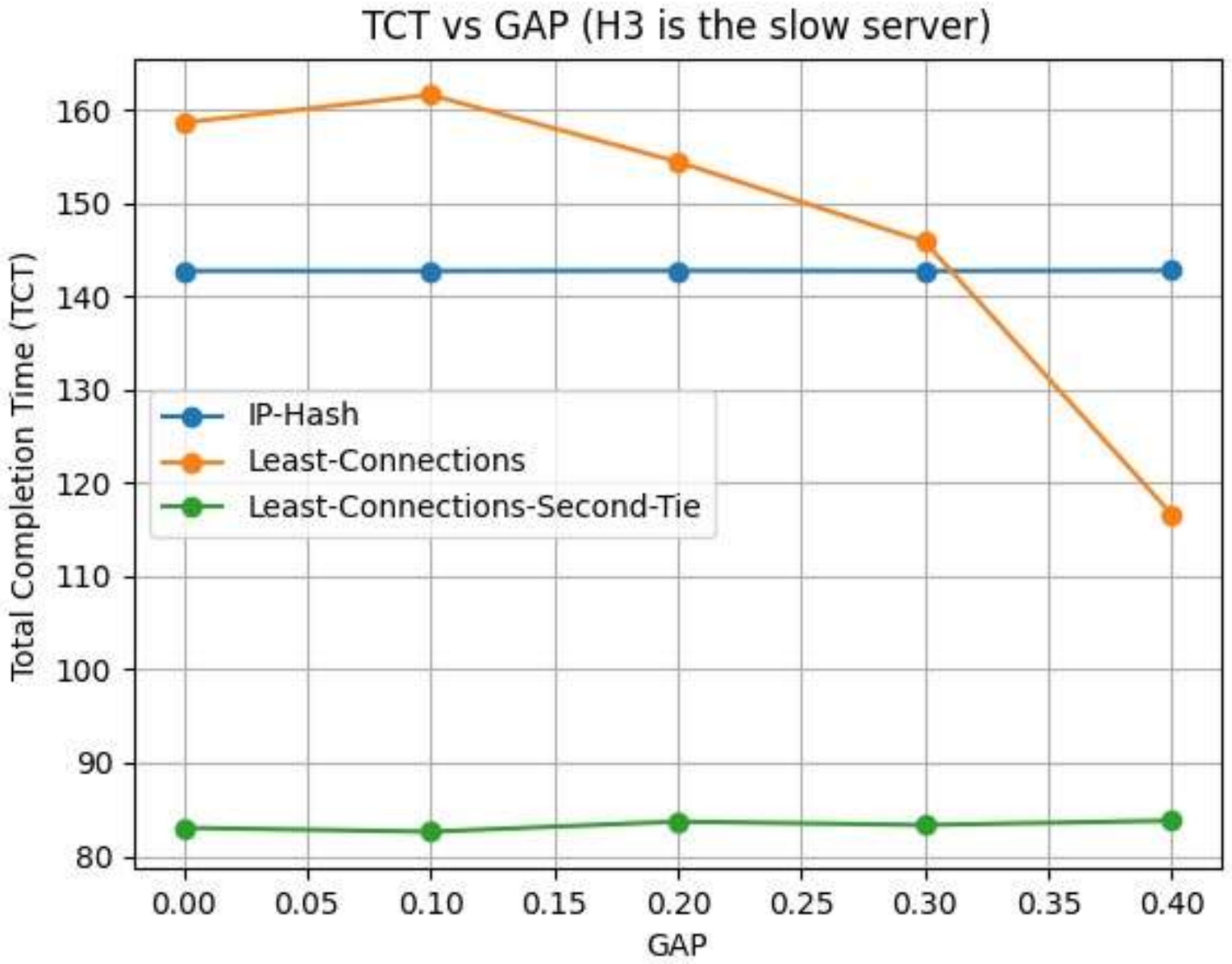



TCT vs GAP (Servers are identical)
IP-Hash
Least-Connections
Least-Connections-Second-Tie
Total Completion Time (TCT)
80
75
70
65
60
55
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
GAP

Client	Hash result	Server
10.0.0.1	1	10.0.0.4
10.0.0.2	0	10.0.0.3
10.0.0.5	1	10.0.0.4



Client	Hash result	Server
10.0.0.1	1	10.0.0.4
10.0.0.2	0	10.0.0.3
10.0.0.5	1	10.0.0.4



Conclusions :-

- Similar servers → there is an advantage for LC.
- Slow + fast server → request distribution became much more important.
- LC usually worked better because it reacts to active connections.
- IP-Hash can win only when the IP mapping sends more traffic to the fast server.
- Tie-breaking matters: when LC has a tie, choosing the fast server improves the result.

References & Appendix

GitHub Repository:

- link

References :

- Load Balancer – GeeksforGeeks
- Load Balancing Algorithms – GeeksforGeeks
- What is IP Hash? | Web Hosting Geeks' Blog
- loadBalancer/loadBalancerLab.py at master · cs236341/loadBalancer
- ICNS3 2025 | ns-3